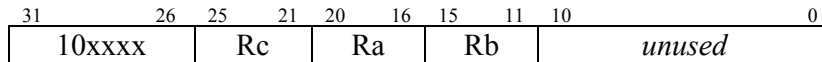


Computation Structures

Instruction Set Architecture Worksheet

Summary of β Instruction Formats

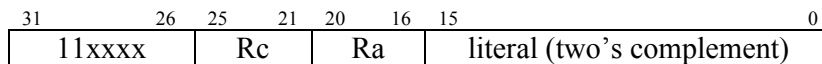
Operate Class:



OP(Ra,Rb,Rc): $\text{Reg[Rc]} \leftarrow \text{Reg[Ra]} \text{ op } \text{Reg[Rb]}$

Opcodes: **ADD** (plus), **SUB** (minus), **MUL** (multiply), **DIV** (divided by)
AND (bitwise and), **OR** (bitwise or), **XOR** (bitwise exclusive or), **XNOR** (bitwise exclusive nor),
CMPEQ (equal), **CMPLT** (less than), **CMPLT** (less than or equal) [result = 1 if true, 0 if false]
SHL (left shift), **SHR** (right shift w/o sign extension), **SRA** (right shift w/ sign extension)

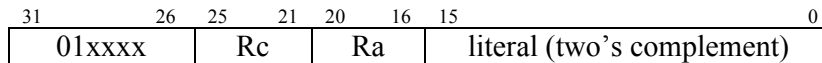
Register	Symbol	Usage
R31	R31	Always zero
R30	XP	Exception pointer
R29	SP	Stack pointer
R28	LP	Linkage pointer
R27	BP	Base of frame pointer



OPC(Ra,literal,Rc): $\text{Reg[Rc]} \leftarrow \text{Reg[Ra]} \text{ op } \text{SEXT(literal)}$

Opcodes: **ADD**C (plus), **SUB**C (minus), **MUL**C (multiply), **DIV**C (divided by)
ANDC (bitwise and), **OR**C (bitwise or), **XOR**C (bitwise exclusive or), **XNOR**C (bitwise exclusive nor)
CMPEQC (equal), **CMPLT**C (less than), **CMPLT**C (less than or equal) [result = 1 if true, 0 if false]
SHLC (left shift), **SHR**C (right shift w/o sign extension), **SRA**C (right shift w/ sign extension)

Other:



LD(Ra,literal,Rc): $\text{Reg[Rc]} \leftarrow \text{Mem}[\text{Reg[Ra]} + \text{SEXT(literal)}]$
ST(Rc,literal,Ra): $\text{Mem}[\text{Reg[Ra]} + \text{SEXT(literal)}] \leftarrow \text{Reg[Rc]}$
JMP(Ra,Rc): $\text{Reg[Rc]} \leftarrow \text{PC} + 4; \text{PC} \leftarrow \text{Reg[Ra]}$
BEQ/BF(Ra,label,Rc): $\text{Reg[Rc]} \leftarrow \text{PC} + 4; \text{if } \text{Reg[Ra]} = 0 \text{ then } \text{PC} \leftarrow \text{PC} + 4 + 4 * \text{SEXT(literal)}$
BNE/BT(Ra,label,Rc): $\text{Reg[Rc]} \leftarrow \text{PC} + 4; \text{if } \text{Reg[Ra]} \neq 0 \text{ then } \text{PC} \leftarrow \text{PC} + 4 + 4 * \text{SEXT(literal)}$
LDR(label,Rc): $\text{Reg[Rc]} \leftarrow \text{Mem}[\text{PC} + 4 + 4 * \text{SEXT(literal)}]$

Opcode Table: (*optional opcodes)

2:0 5:3	000	001	010	011	100	101	110	111
000								
001								
010								
011	LD	ST		JMP	BEQ	BNE		LDR
100	ADD	SUB	MUL*	DIV*	CMPEQ	CMPLT	CMPLT	
101	AND	OR	XOR	XNOR	SHL	SHR	SRA	
110	ADD C	SUB C	MUL C*	DIV C*	CMPEQ C	CMPLT C	CMPLT C	
111	AND C	OR C	XOR C	XNOR C	SHL C	SHR C	SRA C	

Problem 1.

An unnamed associate of yours has broken into the computer (a Beta of course!) that 6.004 uses for course administration. He has managed to grab the contents of the memory locations he believes holds the Beta code responsible for checking access passwords and would like you to help discover how the password code works. The memory contents are shown in the table below:

Addr	Contents	Opcode	Rc	Ra	Rb	Assembly
0x100	0xC05F0008	110000	00010	11111	_____	ADD (R ₃₁ , 0x8, R ₂)
0x104	0xC03F0000	110000	00001	11111	_____	ADD (R ₃₁ , 0x0, R ₁)
L2: 0x108	0xE060000F	111000	00011	00000	_____	AND (R ₀ , 0xF, R ₁)
0x10C	0xF0210004	111100	00001	00001	_____	IRAC (R ₁ , 0x4, R ₁)
0x110	0xA4230800	101001	00001	00011	00011	OR (R ₃ , R ₁ , R ₁)
0x114	0xF4000004	111101	00000	00000	_____	SHR (R ₀ , 0x4, R ₀)
0x118	0xC4420001	110001	00010	00010	_____	SUB (R ₂ , 0x1, R ₂)
0x11C	0x73E20002	011100	11111	00010	_____	BEC (R ₂ , L1, R ₃₁)
0x120	0x73FFFFF9	011100	11111	11111	_____	BEC (R ₃₁ , L2, R ₃₁)
0x124	0xA4230800	101001	00001	00011	_____	NOT R _{XCUTD}
L1: 0x128	0x605F0124	011000	00010	11111	_____	LD (R ₃₁ , 0x124, R ₂)
0x12C	0x90211000	100100	00001	00001	00010	STARE (R ₁ , R ₂ , R ₁)

Further investigation reveals that the password is just a 32-bit integer which is in R0 when the code above is executed and that the system will grant access if R1 = 1 after the code has been executed. What "passnumber" will gain entry to the system?

After 8 times loop 0x110 will

reverse value R0 into R1, the code
is 0xA4230800, so you need the passnumber
008D324x0 to gain entry to the system

Problem 2.

- (A) What assembly instruction could a compiler use to implement $y = x * 8$ on the Beta assuming that MUL and MULC are not available? Assume x is in R0 and y is in R1.

Equivalent assembly instruction: SHL(R0, 3, R1)

- (B) Assume that the registers are initialized to: R0=8, R1=10, R2=12, R3=0x1234, R4=24 before execution of each of the following assembly instructions. For each instruction, provide the value of the specified register or memory location. **If your answers are in hexadecimal, make sure to prepend them with the prefix 0x.**

1. SHL(R3, R4, R5) Value of R5: 0x340D0000

2. ADD(R2, R1, R6) Value of R6: 22

3. ADD(R0, ^{R2}2, R7) Value of R7: 20

4. ST(R1, 4, R3) Value stored: 10 at address: 0x1234 + 4 = 0x1238

- (C) A student tries to optimize his Beta assembly program by replacing a line containing

ADDC(R0, 3*4+5, R1)

by

ADDC(R0, 17, R1)

Is the resulting binary program smaller? Does it run faster?

(circle one) Binary program is SMALLER? yes ... no

(circle one) FASTER? yes ... no

- (D) A BR instruction at location 0x1000 branches to 0x2000. If the binary representation for that BR were moved to location 0x1400 and executed there, where will the relocated instruction branch to?

Branch target for relocated BR (in hex): 0x 2400

- (E) A line in an assembly-language program containing “ADDC(R1,2,R3)” is changed to “ADDC(R1,R2,R3)”. Will the modified program behave differently when executed?

Circle best answer: YES ... NO ... CAN'T TELL

Problem 3.

Each of the following programs is loaded into a Beta's main memory starting at location 0 and execution is started with the Beta's PC set to 0. Assume that all registers have been initialized to 0 before execution begins. Please determine the specified values after execution reaches the HALT() instruction and the Beta stops. Write "CAN'T TELL" if the value cannot be determined. Please write all values in hex.

(A) . = 0
 LD(R31,X+4,R1) *R1 → 3*
 SHLC(R1,2,R1) *00112 → 11002*
 LD(R1,X,R2)
 HALT()
 X: LONG(4)
 4 LONG(3)
 8 LONG(2)
 12 LONG(1)
 16 LONG(0)

Value left in R1: 0x 0xL
 Value left in R2: 0x 0x1

(B) . = 0
 LD(R31,X,R0)
 CMOVE(0,R1)
 L: CMPLTC(R0,0,R2)
 BNE(R2,DONE)
 ADDC(R1,1,R1)
 SHLC(R0,1,R0)
 BR(L)
 1 L DONE: HALT()
 20 X: LONG(0x08306352)

Value left in R0: 0x 0x3063520
 Value left in R1: 0x 4
 Value left in R2: 0x 1
 Value assembler assigns to symbol X: 0x 20

(C) . = 0
 LD(R31,Z,R1)
 SHRC(R1,26,R1) *R1 = 110101...*
 Z: CMPLTC(R1,0x3C,R2) *R1 = 00... 110101*
 HALT() *53 < 60*
 → R2 = 1

Value left in R1: 0x 35
 Value left in R2: 0x 1

(D) . = 0
 LD(R31,X,R0)
 CMOVE(0,R1)
 L: ADDC(R1,1,R1)
 SHRC(R0,1,R0)
 2x10 BNE(R0,L,R2)
 HALT()
 . = 0x100
 X: LONG(5)

Value left in R0: 0x 0
 Value left in R1: 0x 3
 Value left in R2: 0x 10+4
 Value assembler assigns to symbol X: 0x 0x100

$$r_{31} = 14$$

$$r_1 = C \quad r_2 = 17 = 1001 \rightarrow 0010$$

(E)

```
. = 0
LD(r31, X, r0)
CMPL(r0, r31, r1)
BNE(r1, L1, r1)
ADDC(r31, 17, r2)
BEQ(r31, L2, r31)
L1: SRAC(r0, 4, r2)
L2: HALT()
```

Value left in R0? 0x 128654321

Value left in R1? 0x C

Value left in R2? 0x 12865432

```
. = 0x1CE8
X: LONG(0x87654321)
```

Value assembler assigns to L1: 0x 14

(F)

```
. = 0
LD(R31, i, R0)
SHLC(R0, 2, R0)
LD(R0, a-4, R1)
HALT()
a: LONG(0xBADBABE)
LONG(0xDEADBEEF)
LONG(0xC0FFEE)
LONG(0x8BADF00D)
i: LONG(3)
```

$$0011 \rightarrow 1100$$

Contents of R0 (in hex): 0x 5

Contents of R1 (in hex): 0x 10FFFF

$$R_0 = 3$$

$$R_0 = 12$$

$$R_1 = 12 + a - 4 = a + 8 = 24$$

(G)

```
. = 0
LD(R31, Z, R1)
SHRC(R1, 16, R2)
Z: SUBC(R2, 0x3C, R3)
HALT()
```

Value left in R1: 0x C4640D3C

Value left in R3: 0x [4]0

Value assembler assigns to symbol Z: 0x 8

(H)

```
. = 0
LD(R31, X, R0)
CMOVE(0, R1)
L: ADDC(R1, 1, R1)
SHRC(R0, 1, R0)
BNE(R0, L, R2)
HALT()
X: LONG(0xDECAF)
```

Value left in R0: 0x 0

Value left in R1: 0x 14

Value left in R2: 0x 14

MIT OpenCourseWare
<https://ocw.mit.edu/>

6.004 Computation Structures
Spring 2017

For information about citing these materials or our Terms of Use, visit: <https://ocw.mit.edu/terms>.